Domain Logic & Decision Engine

Fire Simulation System

27 stycznia 2026

Spis treści

1 Wprowadzenie **4**
1.1 Cel dokumentu . . . 4
1.2 Zakres dokumentu . . . 4
1.3 Przeznaczenie dokumentu . . . 4
1.4 Odniesienia . . . 4

2 Model symulacji pożaru **4**
2.1 Przegląd architektury . . . 4
2.2 Architektura fizyczna . . . 5
2.3 Przepływ danych . . . 6
2.4 Sektor lasu . . . 6
2.4.1 Stany sektora . . . 6
2.4.2 Parametry sektora . . . 7
2.5 Aktualizacja poziomu ognia . . . 7
2.6 Aktualizacja poziomu spalenia . . . 7
2.7 Poziom gaszenia . . . 7
2.8 Wpływ ognia na parametry środowiskowe . . . 8
2.8.1 Temperatura . . . 8
2.8.2 Wilgotność powietrza . . . 8
2.8.3 Stężenie CO2 . . . 8

3 Rozprzestrzenianie się ognia **8**
3.1 Model rozprzestrzeniania . . . 8
3.2 Współczynnik rozprzestrzeniania β . . . 8
3.3 Współczynnik typu sektora α . . . 9
3.4 Współczynnik kierunkowy . . . 9
3.5 Model wiatru . . . 9

4 Baza danych Maps **9**

5 Format zapisu w bazie danych **10**
5.1 Opis pól . . . 11
5.2 API backendu . . . 12
5.3 Główna komunikacja backend-symulator (FFSup-FFSim) . . . 12
5.3.1 Diagram przedstawiający kolejki i przepływ danych pomiędzy Forest protection i fire processing a Forest environments simulator . . . 13
5.3.2 Format danych z czujników . . . 13
5.4 Dane stanu czujników . . . 13
5.4.1 Format danych z kamer wykrywających dym . . . 15
5.4.2 Komunikacja Fire Brigade Analyser – Fire Brigade Simulator . . . 15

5.4.3 Komunikacja Forester Patrol Analyser – Forester Patrol Simulator 17
5.5 Komunikacja z modułem wizualizacji 18
5.5.1 Format danych w komunikacji pomiędzy modułem przetwarzającym a wizualizującym 18

6 Model danych i logika systemu 19
6.1 Model encji 19
6.1.1 Symulator pożarów – Forest Environments Simulator 19
6.1.2 Aplikacja przetwarzająca – Forest Protection & Fire Processing 19
6.2 Algorytmy wykrywania zagrożeń 20
6.3 Moduł reguł monitorowania oraz akcji gaśniczych 20

7 Wytwarzanie logów i monitorowanie 20
7.1 Kategorie logów 20
7.2 Format logów 21
7.2.1 Logi modułu symulacji 21
7.2.2 Logi backendu 22
7.2.3 Logi frontendu 23

8 Bezpieczeństwo i niezawodność 23

9 Algorytm MCTS 23
9.1 Przegląd 23
9.2 Moduł działania 24
9.3 Faza selekcji (Selection) 24
9.4 Faza ekspansji (Expansion) 24
9.5 Faza symulacji (Simulation) 24
9.6 Faza propagacji wstecznej (Backpropagation) 25
9.7 Strategie nagród 25
9.7.1 Komponenty nagród i kar 25
9.7.2 Strategia agresywna 25
9.7.3 Strategia zrównoważona 25
9.7.4 Strategia konserwatywna 25
9.8 Obliczanie nagrody 26
9.9 Warunki terminalne 26
9.10 Ograniczenia obliczeniowe 26

10 Heurystyki decyzyjne 26
10.1 Heurystyka dla patroli leśnych 26
10.2 Heurystyka wymaganych brygad 26

11 Optymalizacje 27
11.1 Ograniczenie przestrzeni akcji 27
11.2 Klonowanie stanu 27

12 Ograniczenia modelu 27
12.1 Założenia upraszczające 27
12.2 Ograniczenia obliczeniowe 27
12.3 Charakter symulacyjny 27

13 Architektura decyzyjna **27**
13.1 Separacja logiki decyzyjnej . . . 27
13.2 Strategie decyzyjne . . . 28
13.3 LLM jako strategia decyzyjna . . . 28
13.3.1 Architektura LLMAgentBrain . . . 28
13.3.2 Przepływ informacji w LLM . . . 28
13.3.3 Klient LLM i Konfiguracja . . . 28

14 Przepływ decyzyjny **29**
14.1 Cykl decyzyjny . . . 29
14.2 Moduły odpowiedzialne . . . 29

15 Infrastruktura Systemowa **29**
15.1 Komunikacja (RabbitMQ) . . . 29
15.2 Zarządzanie Stanem . . . 29

16 Przyszłe rozszerzenia **30**
16.1 Planowane ulepszenia algorytmów . . . 30
16.2 Rozwój Modelu Językowego . . . 30

17 Podsumowanie **30**

1 Wprowadzenie

1.1 Cel dokumentu

Dokument ten przeznaczony jest dla deweloperów, zespołów R&D oraz osób pracujących nad algorytmami. Zawiera szczegółowy opis logiki domenowej, algorytmów decyzyjnych, modeli symulacji oraz heurystyk wykorzystywanych w systemie. Dokument **nie zawiera** informacji o wdrożeniach ani interfejsie użytkownika.

1.2 Zakres dokumentu

Dokument opisuje:

- Model fizyczny symulacji pożaru
- Algorytm MCTS (Monte Carlo Tree Search)
- Heurystyki decyzyjne
- Model rozprzestrzeniania się ognia
- System nagród i kar w MCTS
- Ograniczenia i założenia modelu

1.3 Przeznaczenie dokumentu

Dokument przeznaczony jest dla zespołu rozwojowego i utrzymaniowego, przyszłych kontynuatorów projektu, a także dla integratorów systemów ForestFire oraz audytorów technicznych.

1.4 Odniesienia

- Artykuły R. Klimek, AGH UST
- Rezultaty dotychczas wytworzonych projektów studenckich

2 Model symulacji pożaru

2.1 Przegląd architektury

Symulator pożaru lasu modeluje fizyczne procesy związane z pożarem, w tym:

- Rozprzestrzenianie się ognia między sektorami
- Zmiany poziomu ognia w sektorze
- Proces spalania (burn level)
- Proces gaszenia przez brygady pożarnicze
- Wpływ warunków środowiskowych (wiatr, wilgotność, temperatura)

Ważne: Symulator działa w module **FFSim** i jest oddzielony od systemu decyzyjnego. Wszystkie decyzje podejmowane są przez system (FFBackend, FFSup), a symulator jedynie wykonuje komendy i symuluje fizykę.

System *ForestFire* został zaprojektowany jako aplikacja rozproszona. Składa się z następujących komponentów:

Komunikacja pomiędzy modułami odbywa się z wykorzystaniem protokołów *WebSocket*/RPC oraz brokera wiadomości *RabbitMQ*. Zarys architektury systemu przedstawiono na Rys. 1.

Tabela 1: Komponenty systemu ForestFire

Komponent	**Funkcja**	**Technologia**
FFSup	Główny moduł analityczny i decyzyjny systemu monitorującego	Java, Python, Flask
FFVis	Interfejs wizualizacyjny (panel webowy), służący zarówno dla FFSup, jak i FFConf	JavaScript, Electron
FFConf	Moduł konfiguracji środowiska i sektorów (edycja map, punktów, stacji pomiarowych itd.)	Python, FastAPI
FFSim	Symulator zdarzeń w monitorowanym obszarze oraz transmisji danych; działa niezależnie, możliwe jest uruchomienie wielu symulatorów	Python, Flask
Baza danych	Przechowywanie danych geoprzestrzennych oraz definicji monitorowanych obszarów i sektorów	MongoDB

2.2 Architektura fizyczna

System uruchamiany jest w kontenerach *Docker*. Odpowiednio przygotowany skrypt umożliwia uruchomienie całego systemu przy pomocy jednego polecenia:

```
docker compose -f docker-compose.yml up --build
```

Każdy moduł może być wdrożony niezależnie, przy pomocy polecenia:

```
docker compose -f docker-compose.yml up --build "<nazwa_modułu>"
```

Gdzie `<nazwa_modułu>` to jedna z następujących wartości:

- `rabbitmq-service`
- `forest-simulator-service`
- `mongo`
- `fire-backend-service`
- `fire-configuration-service`
- `fire-visualization-service`

W celu poprawnego działania modułu `fire-visualization-service` konieczne jest wcześniejsze posiadanie skonfigurowanego klucza *Google API* oraz zainstalowanie zależności środowiska *Electron* za pomocą polecenia:

```
npm install
```

Polecenie to należy wywołać w głównym k

2.3 Przepływ danych

Symulator (FFSim)

Symulator (FFSim) generuje dane o sytuacji w sektorach i warunkach pogodowych. Wybiera w sposób losowy sektor, w którym rozpocznie się pożar, a następnie rozprzestrzenia go po pobliskich sektorach, biorąc pod uwagę czynniki środowiskowe, takie jak prędkość i kierunek wiatru.

Symuluje działania strażaków i patroli leśników po odebraniu poleceń z backendu (FFSup).

Generuje dane z czujników i kamer, a także stan otoczenia, w którym znajdują się patrolujący leśnicy.

Oblicza rekomendacje rozkazów dla straży pożarnej za pomocą algorytmu Monte Carlo Tree Search, a następnie przesyła je do backendu (FFSup). Przy odpowiednim ustawieniu na frontendzie (FFVis), może automatycznie stosować rekomendacje, bez oczekiwania na zgodę z backendu (FFSup).

Backend (FFSup)

Backend (FFSup) przekazuje dane do wyświetlenia we frontendzie (FFVis), a także przekazuje polecenia dla strażaków i patroli leśników do symulatora (FFSim).

Frontend (FFVis)

Frontend (FFVis) pobiera dane z backendu (FFSup) i wizualizuje je na mapie. Drugą rolą frontend jest pobieranie danych z konfiguratora (FFConf) i wizualizacja procesów edycyjnych monitorowanych obszarów.

Administrator (FFConf)

Administrator (FFConf) – tylko ten moduł może definiować i modyfikować konfigurację map i punktów infrastruktury oraz sektorów leśnych.

Maps (Sectors)

Maps (Sectors) to główna baza danych zawierająca definicje obszarów monitorowanych (mapa podkładowa, sektory, inne). Przechowywane w niej są pliki konfiguracyjne mapy o strukturze opisanej w formacie JSON.

2.4 Sektor lasu

2.4.1 Stany sektora

Sektor może znajdować się w jednym z trzech stanów:

- **INACTIVE** – brak ognia
- **ACTIVE** – aktywny pożar
- **LOST** – sektor całkowicie spalony (burn level $\geq$ 100%)

2.4.2 Parametry sektora

- **fireLevel** (0-100): Intensywność ognia
- **burnLevel** (0-100): Procent spalenia sektora
- **extinguishLevel** (0-100): Postęp gaszenia
- **temperature**: Temperatura w stopniach Celsjusza
- **airHumidity**: Wilgotność powietrza (0-100%)
- **windSpeed**: Prędkość wiatru (m/s)
- **windDirection**: Kierunek wiatru (0-360 stopni)
- **litterMoisture**: Wilgotność ściółki (0-100%)
- **co2Concentration**: Stężenie CO2 (ppm)
- **pm2_5Concentration**: Stężenie PM2.5 (μg/m^3)

2.5 Aktualizacja poziomu ognia

Poziom ognia w sektorze jest aktualizowany w każdym ticku symulacji zgodnie z formułą:

$$fireLevel_{t+1} = \min(fireLevel_t + \frac{fireLevel_t}{10} \cdot \alpha \cdot F - extinguishLevel_t, 100) \tag{1}$$

gdzie:

- α – współczynnik typu sektora (zob. sekcja 3.3)
- F – stała mnożnika poziomu ognia (domyślnie 1.0)
- $extinguishLevel_t$ – poziom gaszenia w chwili t

Jeśli $fireLevel_{t+1} \leq 0$, sektor przechodzi w stan **INACTIVE**.

2.6 Aktualizacja poziomu spalenia

Poziom spalenia (burn level) jest aktualizowany zgodnie z formułą:

$$burnLevel_{t+1} = \min(burnLevel_t + 0.00005 \cdot fireLevel_t^3, 100) \tag{2}$$

Jeśli $burnLevel_{t+1} \geq 100$, sektor przechodzi w stan **LOST** i $fireLevel$ jest ustawiany na 0.

2.7 Poziom gaszenia

Poziom gaszenia zależy od liczby brygad pożarniczych w sektorze:

$$extinguishLevel_t = n_{brigades} \cdot M \tag{3}$$

gdzie:

- $n_{brigades}$ – liczba brygad pożarniczych w sektorze
- M – stała mnożnika gaszenia (domyślnie 1.0)

2.8 Wpływ ognia na parametry środowiskowe

2.8.1 Temperatura

Temperatura w sektorze jest aktualizowana zgodnie z formułą:

$$T_{t+1} = T_t + (T_{target} - T_t) \cdot 0.5 - T_t \cdot 0.02 + \epsilon \tag{4}$$

gdzie:

- $T_{target} = T_{initial} + fireLevel \cdot 0.5$ – docelowa temperatura
- $\epsilon \sim U(-0.5, 0.5)$ – losowa zmiana
- $T_{initial}$ – początkowa temperatura sektora

Temperatura jest ograniczona do zakresu $[10, T_{initial} + 80]$.

2.8.2 Wilgotność powietrza

$$H_{t+1} = \max(H_t - fireLevel \cdot 0.4 + \epsilon, 5) \tag{5}$$

gdzie $\epsilon \sim U(-2, 2)$.

2.8.3 Stężenie CO2

$$CO2_{t+1} = \max(CO2_t + fireLevel^{1.1} - CO2_t \cdot 0.01 + \epsilon, 300) \tag{6}$$

gdzie $\epsilon \sim U(-5, 5)$.

3 Rozprzestrzenianie się ognia

3.1 Model rozprzestrzeniania

Ogień rozprzestrzenia się z sektora aktywnego do sąsiednich sektorów z prawdopodobieństwem zależnym od:

- Kierunku i prędkości wiatru
- Typu sektora docelowego
- Kierunku geograficznego względem wiatru

3.2 Współczynnik rozprzestrzeniania β

Prawdopodobieństwo zapalenia się sąsiedniego sektora obliczane jest jako:

$$\beta = \min(\frac{windSpeed}{40} \cdot \alpha_{target} \cdot directionCoef, 1) \tag{7}$$

gdzie:

- $windSpeed$ – prędkość wiatru (m/s)
- α_{target} – współczynnik typu sektora docelowego (zob. sekcja 3.3)
- $directionCoef$ – współczynnik kierunkowy (zob. sekcja 3.4)

3.3 Współczynnik typu sektora α

Współczynnik α zależy od typu sektora:

- **FOREST:** $\alpha = 1.0$ (najwyższe ryzyko)
- **GRASS:** $\alpha = 0.8$
- **BUILDING:** $\alpha = 0.3$
- **WATER:** $\alpha = 0.0$ (niepalne)

3.4 Współczynnik kierunkowy

Współczynnik kierunkowy zależy od różnicy między kierunkiem wiatru a kierunkiem do sąsiedniego sektora:

$$diff = \min(|windDirection - sectorDirection|, 8 - |windDirection - sectorDirection|) \quad (8)$$

Wartości współczynnika:

- $diff = 0$: $directionCoef = 0.8$
- $diff = 1$: $directionCoef = 0.5$
- $diff = 2$: $directionCoef = 0.25$
- $diff = 3$: $directionCoef = 0.1$
- $diff = 4$: $directionCoef = 0.05$
- $diff > 4$: $directionCoef = 0.0$

3.5 Model wiatru

Wiatr jest modelowany jako dynamiczny parametr, który zmienia się w czasie:

- **Prędkość wiatru:** $windSpeed_{t+1} = \max(0, windSpeed_t + \epsilon)$ gdzie $\epsilon \sim U(-2.5, 2.5)$
- **Kierunek wiatru:** zmienia się losowo o -1, 0 lub +1 jednostkę w każdym ticku (cyklicznie, 1-8)

4 Baza danych Maps

Nierelacyjna baza MongoDB przechowuje pliki konfiguracyjne map w formacie JSON. Zawierają one:

- Identyfikator i nazwę lasu
- Ilość wierszy i kolumn w podziale lasu na sektory
- Współrzędne punktów będących wierzchołkami prostokąta reprezentującego las
- Listę czujników wraz ze specyfikacją każdego z nich
- Listę kamer wraz ze specyfikacją każdego z nich
- Listę brygad strażackich wraz z danymi na temat każdej z nich

- Listę patroli leśników wraz z danymi na temat każdego z nich

Dla każdego sektora zapisane są dodatkowo:

- Unikalny identyfikator sektora
- Numer wiersza i kolumny, w których znajduje się sektor
- Typ sektora
- Opis początkowych warunków panujących w sektorze, np. temperatura czy wilgotność
- Granice geograficzne sektora, wyrażone w postaci tablicy zawierającej 4 wierzchołki sektora w formacie $[longitude, latitude]$

5 Format zapisu w bazie danych

Przykładowy zapis JSON:

```
{
   "forestId": "string",
   "forestName": "string",
   "rows": "number",
   "columns": "number",
   "location": [
      {"longitude": "number", "latitude": "number"},
      {"longitude": "number", "latitude": "number"},
      {"longitude": "number", "latitude": "number"},
      {"longitude": "number", "latitude": "number"}
   ],
   "sectors": [
      {
         "sectorId": "number",
         "row": "number",
         "column": "number",
         "sectorType": "enum(SECTOR_TYPE)",
         "initialState": {
            "temperature": "number",
            "windSpeed": "number",
            "windDirection": "enum(GEOGRAPHIC_DIRECTION)",
            "airHumidity": "number",
            "plantLitterMoisture": "number",
            "co2Concentration": "number",
            "pm2_5Concentration": "number"
         },
         "contours": ["number", "number"]
      }
   ],
   "sensors": [
      {
         "sensorId": "number",
         "sensorType": "enum(SENSOR_TYPE)",
         "location": {"longitude": "number", "latitude": "number"},
         "timestamp": "datetime"
```

```
      }
   ],
   "cameras": [
      {
         "cameraId": "number",
         "location": {"longitude": "number", "latitude": "number"},
         "range": "number",
         "timestamp": "datetime"
      }
   ],
   "fireBrigades": [
      {
         "fireBrigadeId": "number",
         "timestamp": "datetime",
         "state": "enum(FIREBRIGADE_STATE)",
         "baseLocation": {"longitude": "number", "latitude": "number"},
         "currentLocation": {"longitude": "number", "latitude": "number"}
      }
   ],
   "foresterPatrols": [
      {
         "foresterPatrolId": "number",
         "timestamp": "datetime",
         "state": "enum(FORESTERPATROL_STATE)",
         "baseLocation": {"longitude": "number", "latitude": "number"},
         "currentLocation": {"longitude": "number", "latitude": "number"}
      }
   ]
}
```

5.1 Opis pól

- **forestId** – identyfikator konfiguracji, czyli nazwa lasu.
- **forestName** – nazwa opisowa lasu.
- **rows, columns** – ilość wierszy i kolumn, na które zostanie podzielony obszar lasu.
- **location** – współrzędne geograficzne czterech wierzchołków obszaru lasu.
- **sectors** – lista konfiguracji każdego sektora:
  - **sectorId** – unikalny identyfikator sektora.
  - **row, column** – położenie sektora w obrębie lasu, wartości całkowite od 1 do odpowiednio rows i columns.
  - **sectorType** – typ sektora; wartość wyliczeniowa SECTOR_TYPE.
  - **initialState** – początkowe warunki w sektorze:
    - **temperature** – temperatura [°C]
    - **windSpeed** – prędkość wiatru [m/s]
    - **windDirection** – kierunek wiatru, wartość wyliczeniowa GEOGRAPHIC_DIRECTION
    - **airHumidity** – wilgotność powietrza [
    - **plantLitterMoisture** – wilgotność ściółki [

    - **co2Concentration** – stężenie CO_2 [ppm]
    - **pm2_5Concentration** – stężenie PM2,5 [µg/m^3]
  - **contours** – granice geograficzne sektora, tablica 4 wierzchołków [$longitude, latitude$]

- **sensors** – lista czujników:
  - **sensorId** – unikalny identyfikator
  - **sensorType** – typ czujnika, wartość wyliczeniowa SENSOR_TYPE
  - **location** – współrzędne geograficzne
  - **timestamp** – stempel czasowy
- **cameras** – lista kamer:
  - **cameraId** – unikalny identyfikator
  - **location** – współrzędne geograficzne
  - **range** – zasięg kamery
  - **timestamp** – stempel czasowy
- **fireBrigades** – lista brygad strażackich:
  - **fireBrigadeId** – unikalny identyfikator
  - **timestamp** – stempel czasowy
  - **state** – aktualna akcja, wartość wyliczeniowa FIREBRIGADE_STATE
  - **baseLocation** – współrzędne bazy strażackiej
  - **currentLocation** – aktualne współrzędne brygady
- **foresterPatrols** – lista patroli leśników:
  - **foresterPatrolId** – unikalny identyfikator
  - **timestamp** – stempel czasowy
  - **state** – aktualna akcja, wartość wyliczeniowa FORESTERPATROL_STATE
  - **baseLocation** – współrzędne bazy grupy leśników
  - **currentLocation** – aktualne współrzędne grupy

5.2 API backendu

Istnieje jeden endpoint HTTP - /run-simulation?interval=<interval>, który pozwala utworzyć subskrypcję na Server Sent Events (SSE) i otrzymywać wiadomości, które wysyłane są z zadanym interwałem czasowym i zawierają aktualny stan symulacji.

5.3 Główna komunikacja backend-symulator (FFSup-FFSim)

Symulator (FFSim) przesyła dane w formacie JSON za pośrednictwem systemu kolejkowego RabbitMQ. Wiadomości te są odbierane przez backend (FFSup) i zawierają następujące informacje:

- stan ekip gaśniczych,
- stan patroli leśników,
- dane z czujników środowiskowych,
- dane z kamer monitorujących obszar lasu.

5.3.1 Diagram przedstawiający kolejki i przepływ danych pomiędzy Forest protection i fire processing a Forest environments simulator

5.3.2 Format danych z czujników

5.4 Dane stanu czujników

Dane stanu czujników są publikowane do odpowiednich kolejek brokera RabbitMQ w zależności od typu czujnika. Każdy komunikat posiada część wspólną, identyczną dla wszystkich typów czujników.

Część wspólna komunikatu czujnika

```
{
  "sensorId": "number",
  "sensorType": "enum(SENSOR_TYPE)",
  "timestamp": "datetime",
  "location": {
    "longitude": "number",
    "latitude": "number"
  },
  "data": {}
}
```

Czujnik temperatury i wilgotności powietrza

```
{
  "sensorId": "number",
  "timestamp": "datetime",
  "sensorType": "SENSOR_TYPE.TEMPERATURE_AND_AIR_HUMIDITY",
  "location": {
    "longitude": "number",
    "latitude": "number"
  },
  "data": {
    "temperature": "number",
    "airHumidity": "number"
  }
}
```

Czujnik prędkości wiatru

```
{
  "sensorId": "number",
  "timestamp": "datetime",
  "sensorType": "SENSOR_TYPE.WIND_SPEED",
  "location": {
    "longitude": "number",
    "latitude": "number"
  },
  "data": {
    "windSpeed": "number"
  }
}
```

Czujnik kierunku wiatru

```
{
  "sensorId": "number",
  "timestamp": "datetime",
  "sensorType": "SENSOR_TYPE.WIND_DIRECTION",
  "location": {
    "longitude": "number",
    "latitude": "number"
  },
  "data": {
    "windDirection": "enum(GEOGRAPHIC_DIRECTION)"
  }
}
```

Czujnik wilgotności ściółki leśnej

```
{
  "sensorId": "number",
  "timestamp": "datetime",
  "sensorType": "SENSOR_TYPE.LITTER_MOISTURE",
  "location": {
    "longitude": "number",
    "latitude": "number"
  },
  "data": {
    "plantLitterMoisture": "number"
  }
}
```

Czujnik stężenia pyłów PM2.5

```
{
  "sensorId": "number",
  "timestamp": "datetime",
  "sensorType": "SENSOR_TYPE.PM2_5",
  "location": {
    "longitude": "number",
    "latitude": "number"
  },
  "data": {
    "pm2_5Concentration": "number"
  }
}
```

Czujnik stężenia dwutlenku węgla CO_2

```
{
  "sensorId": "number",
  "timestamp": "datetime",
  "sensorType": "SENSOR_TYPE.CO2",
  "location": {
    "longitude": "number",
```

```
    "latitude": "number"
  },
  "data": {
    "co2Concentration": "number"
  }
}
```

5.4.1 Format danych z kamer wykrywających dym

Dane stanu kamery

```
{
  "cameraId": "number",
  "timestamp": "datetime",
  "location": {
    "longitude": "number",
    "latitude": "number"
  },
  "data": {
    "smokeDetected": "boolean",
    "smokeLevel": "number",
    "smokeLocation": {
      "longitude": "number",
      "latitude": "number"
    }
  }
}
```

5.4.2 Komunikacja Fire Brigade Analyser – Fire Brigade Simulator

Kierunek: Fire Brigade Analyser → Fire Brigade Simulator

Wysłanie brygady strażackiej do pożaru

```
{
  "fireBrigadeId": "number",
  "action": "FIREBRIGADE_ACTION.EXSTINGUISH",
  "fireState": "enum(FIRE_STATE)",
  "timestamp": "datetime",
  "location": {
    "longitude": "number",
    "latitude": "number"
  }
}
```

Wysłanie brygady strażackiej do bazy

```
{
  "fireBrigadeId": "number",
  "action": "FIREBRIGADE_ACTION.GO_TO_BASE",
  "timestamp": "datetime"
}
```

Lokalizacja bazy strażackiej jest pomijana ze względu na wiedzę posiadaną przez każdą brygadę strażacką.

Kierunek: Fire Brigade Simulator → Fire Brigade Analyser

Informacja o przemieszczaniu się strażaków

```
{
  "fireBrigadeId": "number",
  "state": "FIREBRIGADE_STATE.TRAVELLING",
  "timestamp": "datetime",
  "location": {
    "longitude": "number",
    "latitude": "number"
  }
}
```

Informacja o gaszeniu pożaru

```
{
  "fireBrigadeId": "number",
  "state": "FIREBRIGADE_STATE.EXTINGUISHING",
  "timestamp": "datetime",
  "location": {
    "longitude": "number",
    "latitude": "number"
  }
}
```

Informacja o zakończeniu akcji gaśniczej

```
{
  "fireBrigadeId": "number",
  "state": "FIREBRIGADE_STATE.AVAILABLE",
  "fireState": "FIRE_STATE.EXTINGUISHED",
  "timestamp": "datetime",
  "location": {
    "longitude": "number",
    "latitude": "number"
  }
}
```

Informacja o dostępności brygady

```
{
  "fireBrigadeId": "number",
  "state": "FIREBRIGADE_STATE.AVAILABLE",
  "timestamp": "datetime",
  "location": {
    "longitude": "number",
    "latitude": "number"
  }
}
```

5.4.3 Komunikacja Forester Patrol Analyser – Forester Patrol Simulator

Kierunek: Forester Patrol Analyser → Forester Patrol Simulator

Wysłanie grupy leśników na patrol

```
{
  "foresterPatrolId": "number",
  "action": "FORESTERPATROL_ACTION.PATROL",
  "timestamp": "datetime",
  "location": {
    "longitude": "number",
    "latitude": "number"
  }
}
```

Wysłanie grupy leśników do bazy

```
{
  "foresterPatrolId": "number",
  "action": "FORESTERPATROL_ACTION.GO_TO_BASE",
  "timestamp": "datetime"
}
```

Lokalizacja bazy jest pomijana ze względu na wiedzę posiadaną przez każdą grupę leśników.

Kierunek: Forester Patrol Simulator → Forester Patrol Analyser

Informacja o przemieszczaniu się grupy leśników

```
{
  "foresterPatrolId": "number",
  "state": "FORESTERPATROL_STATE.TRAVELLING",
  "timestamp": "datetime",
  "location": {
    "longitude": "number",
    "latitude": "number"
  }
}
```

Informacja o stanie patrolowanego sektora

```
{
  "foresterPatrolId": "number",
  "state": "FORESTERPATROL_STATE.PATROLLING",
  "timestamp": "datetime",
  "location": {
    "longitude": "number",
    "latitude": "number"
  },
  "sectorState": ""
}
```

Informacja o dostępności grupy leśników

```
{
  "foresterPatrolId": "number",
  "state": "FORESTERPATROL_STATE.AVAILABLE",
  "timestamp": "datetime",
  "location": {
    "longitude": "number",
    "latitude": "number"
  }
}
```

5.5 Komunikacja z modułem wizualizacji

Moduł wizualizacji (FFVis) pełni rolę usługową dla backendu (FFSup) oraz modułu konfiguracyjnego (FFConf). Komunikacja realizowana jest za pomocą dwukierunkowego połączenia pomiędzy serwisem backendowym napisanym w języku Java z wykorzystaniem frameworku Spring Boot a jednym z procesów aplikacji opartej o framework ElectronJS.

5.5.1 Format danych w komunikacji pomiędzy modułem przetwarzającym a wizualizującym

Opis pól w poniższej strukturze danych w formacie JSON jest zgodny z opisem pól zaprezentowanym dla konfiguracji mapy w bazie danych (sekcja 3.2), dlatego nie został tutaj powielony.

```
{
  "forestName": "string",
  "timestamp": "datetime",
  "sectors": [
    {
      "sectorId": "number",
      "state": {
        "timestamp": "datetime",
        "temperature": "number",
        "windSpeed": "number",
        "windDirection": "enum(GEOGRAPHIC_DIRECTION)",
        "airHumidity": "number",
        "plantLitterMoisture": "number",
        "co2Concentration": "number",
        "pm2_5Concentration": "number",
        "threatLevel": "enum(THREAT_LEVEL)",
        "fireState": "enum(FIRE_STATE)"
      }
    }
  ],
  "fireBrigades": [
    {
      "fireBrigadeId": "number",
      "sectorId": "number",
      "location": {
        "longitude": "number",
        "latitude": "number"
      },
```

```
      "state": "enum(FIREBRIGADE_STATE)",
      "action": "enum(FIREBRIGADE_ACTION)"
    }
  ],
  "foresterPatrols": [
    {
      "foresterPatrolId": "number",
      "timestamp": "datetime",
      "state": "enum(FORESTERPATROL_STATE)",
      "baseLocation": {
        "longitude": "number",
        "latitude": "number"
      },
      "currentLocation": {
        "longitude": "number",
        "latitude": "number"
      }
    }
  ]
}
```

6 Model danych i logika systemu

6.1 Model encji

Każdy czujnik posiada unikalny identyfikator, położenie geograficzne oraz wielkość fizyczną, którą mierzy. W zależności od rodzaju mierzonej wielkości, pod kluczem `data` może znajdować się jedna wartość (np. w przypadku czujnika stężenia CO_2 — stężenie CO_2) lub wiele wartości (np. w przypadku czujnika temperatury i wilgotności powietrza — temperatura oraz wilgotność).

Jednostki wykonawcze, takie jak brygady straży pożarnej oraz patrole leśników, również posiadają swoje unikalne identyfikatory, aktualne położenie oraz stan operacyjny (np. podróżowanie, gaszenie pożaru, patrolowanie).

6.1.1 Symulator pożarów – Forest Environments Simulator

Struktura modułu *Forest Environments Simulator* odpowiada za symulację środowiska leśnego oraz dynamiki rozprzestrzeniania się pożarów, a także za odwzorowanie działań podejmowanych przez jednostki straży pożarnej i patrole leśników.

Forest Environments Simulator został podzielony na dwa podkomponenty:

- **Sensor Data Generator** – odpowiedzialny za generowanie danych symulujących informacje otrzymywane z rzeczywistych czujników. Moduł ten został podzielony na dalsze podkomponenty, z których każdy odpowiada za konkretny typ czujnika.
- **Fire Brigade Simulator** – odpowiedzialny za odbieranie zleceń akcji strażackich z kolejki *Fire Brigades Action Queue* w brokerze RabbitMQ, symulowanie działań brygad strażackich oraz wysyłanie informacji o stanie brygad do kolejki *Fire Brigade State Queue* w brokerze RabbitMQ.

6.1.2 Aplikacja przetwarzająca – Forest Protection & Fire Processing

Moduł *Forest Protection & Fire Processing* odpowiada za analizę danych środowiskowych oraz koordynację działań jednostek wykonawczych systemu.

Obsługa brygad strażackich Struktura modułu *Forest Protection & Fire Processing* w zakresie obsługi brygad strażackich obejmuje mechanizmy odpowiedzialne za podejmowanie decyzji operacyjnych, generowanie poleceń akcji oraz monitorowanie stanu realizowanych zadań.

6.2 Algorytmy wykrywania zagrożeń

System analizuje dane środowiskowe w sposób cykliczny, w konfigurowalnym interwale czasowym (np. co 5 sekund).

Proces wykrywania oraz modelowania rozprzestrzeniania się pożaru opiera się na probabilistycznym modelu Monte Carlo. Dla każdego sektora znajdującego się w stanie pożaru analizowane są sektory sąsiednie, a prawdopodobieństwo zapalenia się danego sektora obliczane jest za pomocą funkcji `calculate_beta()`, która uwzględnia zmienne pogodowe, ukształtowanie terenu oraz położenie geograficzne sektorów.

Każdy sektor monitoruje swój aktualny stan na podstawie danych pochodzących z przypisanych sensorów.

Detekcja zagrożenia polega na obserwowaniu zmian stanu sektorów, w szczególności przejścia sektora ze stanu `INACTIVE` do stanu `ACTIVE`.

6.3 Moduł reguł monitorowania oraz akcji gaśniczych

Moduł reguł wykorzystuje algorytm *Monte Carlo Tree Search* (MCTS), który uruchamiany jest asynchronicznie w osobnym wątku, w konfigurowalnym interwale czasowym.

Algorytm MCTS analizuje sklonowany stan mapy lasu i generuje rekomendowane akcje w postaci par `(unitId, sectorId)`, reprezentujących przypisanie jednostek gaśniczych do sektorów wymagających interwencji.

Wygenerowane rekomendacje są następnie filtrowane pod kątem dostępności jednostek oraz przesyłane do odpowiednich kolejek komunikacyjnych o wysokim priorytecie.

System obsługuje również akcje manualne, odbierane z kolejek `Forester Patrol Action Queue` oraz `Fire Brigades Action Queue` przez menedżera agentów.

7 Wytwarzanie logów i monitorowanie

7.1 Kategorie logów

System generuje logi w kilku kategoriach, odpowiadających poszczególnym warstwom architektury oraz pełnionym przez nie funkcjom.

Logi aplikacyjne

Logi aplikacyjne obejmują wszystkie zdarzenia zachodzące w backendzie (FFSup), w szczególności:

- obsługę kolejek komunikatów (subskrypcje, otrzymywanie wiadomości, błędy),
- aktualizację stanu brygad gaśniczych oraz patroli leśników,
- obsługę danych pochodzących z czujników i kamer,
- reakcje systemu na zdarzenia środowiskowe,
- wysyłanie poleceń uruchomienia i sterowania symulacją.

Logi te umożliwiają pełne śledzenie przebiegu przetwarzania danych oraz diagnostykę błędów logicznych i komunikacyjnych.

Logi symulatora

Logi symulatora (FFSim) dotyczą w szczególności:

- tworzenia oraz konfiguracji kolejek i tematów RabbitMQ,
- dodawania producentów i konsumentów komunikatów,
- wysyłania i odbierania wiadomości,
- inicjalizacji oraz przebiegu pożarów,
- zmian stanów sektorów oraz ich wygaszania,
- generowanych danych środowiskowych i stanu czujników (np. temperatura, wilgotność).

Logi frontendu

Logi frontendu (FFVis) mają głównie charakter techniczny i obejmują:

- uruchamianie i zamykanie aplikacji (tryb deweloperski i produkcyjny),
- błędy wizualizacji danych,
- odbiór i parsowanie danych otrzymanych z backendu.

Logi systemowe

Logi systemowe obejmują logi generowane przez:

- kontenery Docker,
- bazę danych,
- brokera RabbitMQ,
- technologie użyte do budowy systemu (m.in. Spring Boot, Flask).

Logi są gromadzone przez mechanizmy platformy Docker, która zarządza ich rotacją oraz cyklicznym usuwaniem po przekroczeniu określonej liczby uruchomień kontenera lub maksymalnego rozmiaru plików logów.

7.2 Format logów

System posiada rozbudowany mechanizm logowania. Logi są wypisywane na standardowe wyjście przez wszystkie podsystemy. Ze względu na zastosowanie konteneryzacji, Docker gromadzi logi osobno dla każdego modułu i zarządza nimi zgodnie z domyślnymi ustawieniami platformy.

Szczegółowy format logów oraz ich przykłady zostały przedstawione w załączniku do dokumentacji.

7.2.1 Logi modułu symulacji

Logi powiązane z komunikacją RabbitMQ

- All queues and topics are created and bound.
- Queue created: {queue_name}
- Queue '{queue_name}' bound to topic '{topic_name}'

- Received message {data} from {queue_name}
- Waiting for messages in queue: {queue_name}
- Error consuming messages: {e}
- Retrieved oldest received message: {oldest_message} from queue: {queue_name}
- Message received from Fire brigades queue: {message}
- Channel is None

Logi systemu rekomendacyjnego

- MCTS recommended actions: {recommended_actions}
- Error in MCTS prediction: {str(e)}
- MCTS prediction thread started
- Starting MCTS search with time limit {MAX_SIMULATION_TIME} seconds

Logi związane z poziomem ognia

- Fire started in sector {sector_id}, column: {column}, row: {row}
- New fire level in sector {sector_id} is {fire_level}
- Sector {sector_id} is extinguished

Logi działań brygad gaśniczych

- Order received: Brigade {fire_brigade_id} returning to base
- Order received: Brigade {fire_brigade_id} extinguishing fire at {location}

Logi patroli leśników Logi dotyczące patroli leśników są analogiczne do logów generowanych dla brygad gaśniczych.

Logi stanu czujników i środowiska

- Camera {camera_id} has data: {data}
- Initial wind: {speed} {direction}

7.2.2 Logi backendu

Logi komunikacji RabbitMQ

- Subscribed to sendMessageToQueue {}
- Failed to serialize message: {}
- Failed to declare queue: {}
- Failed to parse RMQ message: {}

Logi stanu środowiska Przykładowy log:

> Wind speed updated for sector {}: {} m/s

Analogiczne logi istnieją dla wszystkich parametrów środowiskowych.

Logi czujników Przykładowy log:

> Sector at location {} not found for temperature and humidity sensor

Analogiczne logi generowane są dla wszystkich typów czujników, np.:

> Litter moisture sensor event received at location: {}

Logi stanu symulacji

- Starting simulation with interval: {} seconds
- Initial state created with {} sectors
- Sending HTTP request to start simulation

Logi brygad gaśniczych

- Fire brigade event received – ID: {}, Location: {}
- Fire brigade state updated – ID: {}

Logi patroli leśników Logi dotyczące patroli leśników są analogiczne do logów generowanych dla brygad gaśniczych.

7.2.3 Logi frontendu

Logi frontendu mają głównie charakter techniczny i obejmują informacje o uruchomieniu i zatrzymaniu aplikacji oraz o odbiorze i przetwarzaniu danych wizualizowanych w interfejsie użytkownika.

8 Bezpieczeństwo i niezawodność

System jest niestabilny. Błędy, takie jak "teleportujące się"lub znikające obiekty występują dość często. Powodują one kolejne błędy, jak np. nieprawidłowe rekomendacje. Momentami można odnieść wrażenie, że na backendzie dzieje się coś innego, niż jest widoczne w warstwie wizualnej. Błędy te wynikają z nieprawidłowej obsługi wątków a także kolejki komunikatów. W bardzo rzadkich sytuacjach możliwe jest duplikowanie obiektów. Niektóre sensory nie działają lub podają dane niezgodne z symulacją.

9 Algorytm MCTS

9.1 Przegląd

Monte Carlo Tree Search (MCTS) jest wykorzystywany w module **FFSup** do generowania rekomendacji decyzyjnych. Algorytm buduje drzewo możliwych stanów systemu, symulując akcje agentów i oceniając nagrody.

9.2 Moduł działania

MCTS działa w module **FFSup** i wykorzystuje następujące wejścia:

- Stan wszystkich sektorów (fireLevel, burnLevel, fireState)
- Pozycje i stany wszystkich brygad pożarniczych
- Pozycje i stany wszystkich patroli leśnych
- Konfiguracja lasu (sąsiedztwo sektorów, typy sektorów)

Wyjścia:

- Lista rekomendowanych akcji: $(agentId, sectorId)$
- Każda rekomendacja określa, którą brygadę wysłać do którego sektora

9.3 Faza selekcji (Selection)

Algorytm wybiera ścieżkę od korzenia do liścia używając formuły UCT (Upper Confidence bound applied to Trees):

$$UCT(node, parent) = \frac{Q[node]}{N[node]} + c \cdot \sqrt{\frac{\ln(N[parent])}{N[node]}} \tag{9}$$

gdzie:

- $Q[node]$ – całkowita nagroda dla węzła
- $N[node]$ – liczba wizyt węzła
- $N[parent]$ – liczba wizyt rodzica
- $c = \sqrt{2}$ – współczynnik eksploracji (dla UCT1)

Węzeł z najwyższą wartością UCT jest wybierany do ekspansji.

9.4 Faza ekspansji (Expansion)

Po dotarciu do liścia, węzeł jest rozwijany przez wygenerowanie wszystkich możliwych akcji:

- Dla każdej dostępnej brygady pożarniczej: możliwe akcje to przypisanie do sektora z aktywnym pożarem
- Dla patroli leśnych: używana jest heurystyka (zob. sekcja 10.1)

9.5 Faza symulacji (Simulation)

Z wybranego węzła wykonywana jest losowa symulacja do stanu terminalnego (lub do maksymalnej liczby kroków). W każdym kroku:

1. Wybierana jest losowa akcja dla każdego agenta
2. Stan systemu jest aktualizowany (rozprzestrzenianie ognia, gaszenie)
3. Sprawdzane jest, czy osiągnięto stan terminalny

9.6 Faza propagacji wstecznej (Backpropagation)

Nagroda z symulacji jest propagowana wstecz do korzenia, aktualizując wartości Q i N dla wszystkich węzłów na ścieżce:

for węzeł w ścieżce (od liścia do korzenia) **do**
  $N[wze] \leftarrow N[wze] + 1$
  $Q[wze] \leftarrow Q[wze] + reward$
end for

9.7 Strategie nagród

System wykorzystuje trzy strategie nagród, które różnią się wagami przypisanymi do różnych komponentów:

9.7.1 Komponenty nagród i kar

- **LOST_SECTOR_PENALTY** – kara za utracone sektory (burn level $\geq$ 100%)
- **EXTINGUISHED_FIRES_REWARD** – nagroda za ugaszone pożary
- **SPREAD_PREVENTION_REWARD** – nagroda za zapobieganie rozprzestrzenianiu
- **BURNED_SECTOR_PENALTY** – kara proporcjonalna do poziomu spalenia
- **SECTOR_FIRE_LEVEL_PENALTY** – kara związana z intensywnością ognia

9.7.2 Strategia agresywna

Wysokie wagi dla:

- EXTINGUISHED_FIRES_REWARD
- SPREAD_PREVENTION_REWARD

Niskie wagi dla kar. Skupia się na szybkim gaszeniu pożarów.

9.7.3 Strategia zrównoważona

Zbalansowane wagi dla wszystkich komponentów. Optymalizuje zarówno gaszenie, jak i zapobieganie rozprzestrzenianiu.

9.7.4 Strategia konserwatywna

Wysokie wagi dla:

- SPREAD_PREVENTION_REWARD
- LOST_SECTOR_PENALTY

Skupia się na ochronie sektorów przed utratą.

9.8 Obliczanie nagrody

Całkowita nagroda dla stanu obliczana jest jako:

$$reward = penalties + rewards \tag{10}$$

gdzie:

$$penalties = -lostSectors \cdot LOST_PENALTY - totalFireIntensity \cdot FIRE_LEVEL_PENALTY - totalBur \tag{11}$$

$$rewards = extinguishedCount \cdot EXTINGUISHED_REWARD + firebreakCount \cdot SPREAD_PREVENT \tag{12}$$

9.9 Warunki terminalne

Symulacja MCTS kończy się, gdy:

- Wszystkie pożary zostały ugaszone (fireState = INACTIVE dla wszystkich sektorów)
- Wszystkie sektory zostały utracone (fireState = LOST dla wszystkich sektorów)
- Osiągnięto maksymalną liczbę kroków symulacji

9.10 Ograniczenia obliczeniowe

- **Czas obliczeń:** domyślnie 1 sekunda na wywołanie MCTS
- **Maksymalna liczba kroków symulacji:** 100 kroków (zabezpieczenie przed nieskończoną pętlą)
- **Liczba workerów MCTS:** konfigurowalna (domyślnie 2)

10 Heurystyki decyzyjne

10.1 Heurystyka dla patroli leśnych

Patrole leśne używają heurystyki zamiast MCTS do wyboru sektorów do patrolowania:

1. **Filtrowanie:** Ignorowane są sektory z aktywnym pożarem (fireState = ACTIVE)
2. **Priorytetyzacja:** Priorytet mają "starsze"sektory – te, które nie były patrolowane najdłużej
3. **Wybór:** Każdy patrol wybiera inny sektor (brak duplikatów)

Wiek sektora obliczany jest jako:

$$age = currentStep - lastPatrolStep \tag{13}$$

Sektory nigdy nie patrolowane traktowane są jako najstarsze.

10.2 Heurystyka wymaganych brygad

Liczba brygad wymaganych do skutecznego gaszenia sektora obliczana jest jako:

$$requiredBrigades = \lceil \frac{fireLevel}{3} \rceil \tag{14}$$

11 Optymalizacje

11.1 Ograniczenie przestrzeni akcji

Aby zmniejszyć przestrzeń akcji w MCTS, system wykorzystuje następujące optymalizacje:

- **TOP_SECTOR_PERCENTAGE:** Rozważane są tylko sektory z najwyższym poziomem ognia (domyślnie top 10%)
- **MAX_COORDINATED_SECTORS:** Maksymalna liczba sektorów, do których można przypisać brygady w jednym kroku (domyślnie 10)

11.2 Klonowanie stanu

Aby uniknąć współdzielenia stanu między węzłami MCTS, wszystkie sektory i agenci są klonowane przed użyciem w symulacji. Zapewnia to poprawność predykcji.

12 Ograniczenia modelu

12.1 Założenia upraszczające

- **Model wiatru:** Uproszczony model losowych zmian, nie uwzględnia rzeczywistych danych meteorologicznych
- **Rozprzestrzenianie ognia:** Prawdopodobieństwo zapalenia zależy tylko od wiatru i typu sektora, nie uwzględnia topografii
- **Gaszenie:** Liniowa zależność między liczbą brygad a poziomem gaszenia
- **Agenty:** Agenty poruszają się z stałą prędkością, nie uwzględniają przeszkód terenowych

12.2 Ograniczenia obliczeniowe

- MCTS działa w czasie rzeczywistym, więc czas obliczeń jest ograniczony
- Symulacja MCTS jest uproszczona w porównaniu do pełnej symulacji fizycznej
- Nie wszystkie możliwe akcje są rozważane (optymalizacje przestrzeni akcji)

12.3 Charakter symulacyjny

Wszystkie dane są generowane przez symulator:

- Brak realnych czujników środowiskowych
- Brak realnych danych meteorologicznych
- Wszystkie parametry są modelowane matematycznie

13 Architektura decyzyjna

13.1 Separacja logiki decyzyjnej

Logika decyzyjna jest oddzielona od agentów. Agent reprezentuje runtime i wykonanie, podczas gdy decyzja jest wymienialnym modułem.

13.2 Strategie decyzyjne

System wspiera następujące strategie decyzyjne:

- **Deterministyczna:** Reguły i heurystyki
- **MCTS:** Monte Carlo Tree Search (obecnie używane)
- **MCTS:** Monte Carlo Tree Search (używane w module wsparcia)
- **LLM-driven:** Strategia oparta na Large Language Models (zaimplementowana w agentach)

13.3 LLM jako strategia decyzyjna

System posiada w pełni zaimplementowaną integrację z modelami LLM (domyślnie `gpt-4o-mini`) poprzez klasę `LLMAgentBrain`. Umożliwia to agentom podejmowanie autonomicznych decyzji na podstawie analizy kontekstu w języku naturalnym.

13.3.1 Architektura LLMAgentBrain

Każdy agent (strażak, patrol) posiada własny moduł `LLMAgentBrain`, który odpowiada za:

1. **Percepcję:** Zbieranie informacji o otoczeniu (sektory z pożarami), stanie własnym agenta oraz działaniach innych agentów (peer announcements).
2. **Kontekst:** Budowanie promptu systemowego i użytkownika. Prompt systemowy definiuje rolę agenta, a prompt użytkownika zawiera aktualny stan świata sformatowany jako tekst.
3. **Decyzję:** Komunikację z API OpenAI (przez `LLMClient`) w celu uzyskania decyzji w formacie JSON.

13.3.2 Przepływ informacji w LLM

Proces decyzyjny LLM wygląda następująco:

- **Input:** Pozycja agenta, stan paliwa/wody, lista aktywnych pożarów, warunki pogodowe (wiatr), ostatnie komunikaty od innych agentów.
- **Przetwarzanie:** Model językowy analizuje sytuację, biorąc pod uwagę odległość do pożarów, ich priorytet oraz działania innych jednostek, aby uniknąć duplikacji pracy.
- **Output:** Struktura JSON zawierająca typ akcji (np. `MOVE`, `EXTINGUISH`) oraz parametry (np. `sectorId`). Model generuje również `reasoning` - uzasadnienie decyzji w języku naturalnym.

13.3.3 Klient LLM i Konfiguracja

Komunikacja odbywa się poprzez asynchronicznego klienta `LLMClient`. System wymaga poprawnej konfiguracji klucza API (`OPENAI_API_KEY`). W przypadku braku dostępu do API, agent nie podejmuje decyzji autonomicznie (fallback do braku akcji lub sterowania ręcznego).

14 Przepływ decyzyjny

14.1 Cykl decyzyjny

1. **Inicjalizacja:** System uruchamia symulację (`SimpleSimulationEngine`).
2. **Symulacja:** Silnik aktualizuje stan świata (pożary, wiatr) i agentów.
3. **Wsparcie (MCTS):** `SupportService` w tle analizuje stan lasu i generuje globalne rekomendacje przy użyciu MCTS, które są wysyłane przez RabbitMQ.
4. **Autonomia (LLM):** Równolegle, agenci posiadający `LLMAgentBrain` okresowo (co ok. 10s) analizują otoczenie i podejmują autonomiczne decyzje o ruchu lub gaszeniu.
5. **Interakcja:** Użytkownik widzi stan i rekomendacje w `FFVis`, mogąc wydawać rozkazy ręczne, które nadpisują autonomię agentów.
6. **Wykonanie:** Silnik aplikuje decyzje (ruch agentów, gaszenie) i aktualizuje fizykę w kolejnych krokach czasowych.

14.2 Moduły odpowiedzialne

- **FFSup:** Generowanie rekomendacji (MCTS, LLM, heurystyki)
- **FFBackend:** Agregacja i publikacja rekomendacji
- **FFVis:** Prezentacja rekomendacji użytkownikowi
- **FFSim:** Wykonanie akcji i aktualizacja stanu fizycznego

15 Infrastruktura Systemowa

15.1 Komunikacja (RabbitMQ)

System wykorzystuje architekturę zdarzeniową opartą o kolejki wiadomości RabbitMQ, co zapewnia luźne powiązanie między modułami symulacji i wsparcia. Kluczowe kanały komunikacji obejmują:

- **support.data.aggregated:** Publikowane przez symulator, zawiera zagregowany stan świata (sektory, agenty) niezbędny dla modułu wsparcia.
- **simulation.recommendations:** Publikowane przez moduł wsparcia, zawiera rekomendacje działań (np. przemieszczenie brygady) wygenerowane przez MCTS.
- **fire_brigade_actions / forester_actions:** Kanały dla konkretnych rozkazów kierowanych do agentów.

15.2 Zarządzanie Stanem

Moduł **FFSup** wykorzystuje klasę `StateManager` do utrzymania spójnego obrazu świata:

1. **Wątkobezpieczeństwo:** Wszystkie operacje odczytu i zapisu są chronione blokadami (locks), co pozwala na równoległe przetwarzanie wiadomości z RabbitMQ i zapytań od workerów MCTS.
2. **Minimalne aktualizacje:** System przetwarza i przechowuje tylko niezbędne zmiany stanu, co optymalizuje wydajność przy dużej liczbie sektorów.

3. **Snapshoty:** Udostępnia mechanizm pobierania pełnej kopii stanu (snapshot) dla algorytmów decyzyjnych, gwarantując, że MCTS i LLM operują na niezmiennych w czasie obliczeń danych.

16 Przyszłe rozszerzenia

16.1 Planowane ulepszenia algorytmów

- **FFDI (Fire Weather Index):** Wprowadzenie wskaźnika pogody pożarowej do modelu rozprzestrzeniania
- **Zaawansowane modele wiatru:** Uwzględnienie rzeczywistych danych meteorologicznych
- **Topografia:** Wpływ ukształtowania terenu na rozprzestrzenianie ognia
- **Dynamiczne priorytety:** Adaptacyjne wagi w strategiach nagród

16.2 Rozwój Modelu Językowego

- **Fine-tuning:** Dostrojenie modelu do specyfiki domeny pożarniczej w celu zmniejszenia liczby tokenów i zwiększenia precyzji.
- **Memory-augmentation:** Wprowadzenie długoterminowej pamięci dla agentów, aby pamiętali historię pożarów w danym sektorze.

17 Podsumowanie

System wykorzystuje zaawansowane algorytmy (MCTS) i heurystyki do generowania rekomendacji decyzyjnych. Kluczowe cechy:

- Separacja symulatora od systemu decyzyjnego
- Hybrydowa logika decyzyjna łącząca globalną optymalizację MCTS z lokalną autonomią agentów opartą o LLM
- Model fizyczny symulacji z uwzględnieniem wiatru i typu sektora
- System nagród i kar z możliwością konfiguracji strategii
- Optymalizacje przestrzeni akcji dla wydajności